

Dosya Sistemleri

I-Node Sonrasi: Disk Bloklari, Dosya Operasyonlari,
FS Yapisi, FAT ve Yedekleme

CSE312 Operating Systems — Lecture 07 | Spring 2026

Kapsam	Sayfa 110+ — I-Node disk bloklari ve sonrasi
Konular	Multi-level Indirection, dosya operasyonlari, FS bilesenler, FAT, Dump/Backup
Hazirlayan	CSE312 Ders Notlari — Spring 2026

İçindekiler

- 1 I-Node'da Disk Blok Bilgisi — Temel Sorun
- 2 Tek Katmanlı Dolaylı Adres (Single Indirect Block)
- 3 Çok Katmanlı Dolaylama (Multi-level Indirection)
- 4 Dosya Açma (open) — Adım Adım
- 5 Dosya Oluşturma (create)
- 6 Dosya Okuma (read)
- 7 Dosya Yazma (write)
- 8 Seek — Konum Degistirme
- 9 Dosya Kapatma (close)
- 10 Link (Hard Link) Oluşturma
- 11 Unlink — Dosya Silme
- 12 Metadata Güncelleme
- 13 Dizin Oluşturma ve Silme
- 14 Rename — Yeniden Adlandırma
- 15 Dosya Sistemi Bileşenleri (Boot, Superblock, I-List, Data)
- 16 Windows FAT Dosya Sistemi
- 17 FAT'ta Uzun Dosya Adları
- 18 Disk Dump ve Yedekleme (Backup)
- 19 Quiz Soruları ve Cevapları
- 20 Kitap Bölüm Sonu Soruları ve Çözümleri

1. I-Node'da Disk Blok Bilgisi — Temel Sorun

Bir i-node'un en kritik görevi, dosyanın içeriğinin diskte **hangi bloklarda** saklandığını kaydetmektir. Bu basit gibi görünen sorun aslında derin bir tasarım ikilemine yol açar:

! Temel İkilem

Kucuk bir blok adresi dizisi tutarsak dosyanın maksimum boyutunu kısıtlarız. Büyük bir dizi tutarsak i-list'teki her i-node buyur, kucuk dosyalar için alan israf ederiz.

Klasik Unix çözümü: **karma (hybrid) bir adres yapısı**. I-node içinde hem doğrudan blok adresleri hem de dolaylı blok işaretçileri bulunur.

Alan	Kapasite (4KB blok, 4B adres)	Disk Erisimi
10 doğrudan blok (direct)	$10 \times 4 \text{ KB} = 40 \text{ KB}$	1 erişim
1 tek dolaylı (single indirect)	$(4096/4) \times 4 \text{ KB} = 4 \text{ MB}$	2 erişim
1 çift dolaylı (double indirect)	$(1024)^2 \times 4 \text{ KB} \approx 4 \text{ GB}$	3 erişim
1 uçlu dolaylı (triple indirect)	$(1024)^3 \times 4 \text{ KB} \approx 4 \text{ TB}$	4 erişim

+ Neden Bu Tasarım Zekice?

Cogu dosya kucuktur: 10 doğrudan blok = 40 KB anlık erişim sağlar ve bu dosyaların büyük cogunlugunu kapsiyar. Büyük dosyalar için dolaylı bloklar devreye girer; bu sayede i-list boyutu minimal tutulur.

2. Tek Katmanlı Dolaylı Adres (Single Indirect Block)

10 doğrudan blok 40 KB'ı geçen dosyalar için yeterli değildir. Bu noktada i-node, bir **tek dolaylı blok (single indirect block)** adresine işaret eder. Bu blok, veri bloklarının adreslerini içeren bir diziye sahiptir.

```

I-Node
direct[0] —————> [4 KB veri]
direct[1] —————> [4 KB veri]
... (10 dogrudan blok = 40 KB)
direct[9] —————> [4 KB veri]
single_indirect —> [adres0][adres1]...[adres1023]
|
v
[4 KB veri blogu]
Kapasite: 4096/4 = 1024 adres x 4 KB = 4 MB
                Single indirect block yapisi (4 KB blok, 4B adres varsayimi)

```

Tek dolaylı blok **2 disk erişimi** gerektirir: önce dolaylı bloğu oku (adres listesini al), sonra veri bloğunu oku.

3. Çok Katmanlı Dolaylama (Multi-level Indirection)

"Yazılım mühendisliğindeki her sorun bir dolaylı katman eklenerek çözülebilir." — David Wheeler

4 MB yeterli olmadığında çift dolaylı blok devreye girer:

3.1 Çift Dolaylı Blok (Double Indirect)

```

I-Node
double_indirect
|
v
[adres0][adres1]...[adres1023] <- çift dolaylı blok (1024 adres)
|
v
[adres0][adres1]...[adres1023] <- tek dolaylı blok (1024 adres)
|
v
[4 KB veri blogu]
Kapasite: 1024 x 1024 x 4 KB = 4 GB
Disk erişimi: 3 (çift-ind. blok + tek-ind. blok + veri blogu)

```

3.2 Uçlu Dolaylı Blok (Triple Indirect)

```
triple_indirect
|→ [1024 adres] <- uclu dolayli
|→ [1024 adres] <- cift dolayli
|→ [1024 adres] <- tek dolayli
|→ [4 KB veri]
Kapasite:  $1024^3 \times 4 \text{ KB} \approx 4 \text{ TB}$ 
Disk erisimi: 4
```

Üçlü dolaylı blok bazı sistemlerde bulunur. 4KB blok boyutuyla pratik maksimum dosya boyutu ~4 TB'dır — bugünkü büyük dosyalar için yeterlidir.

3.3 Tam Kapasite Hesabi

```
# 4KB blok, 4B adres (1 blok = 4096/4 = 1024 adres)
dogrudan = 10 * 4096 = 40,960 B (~40 KB)
single_ind = 1024 * 4096 = 4,194,304 B (~4 MB)
double_ind = 1024 * 1024 * 4096 = 4,294,967,296 B (~4 GB)
triple_ind = 1024 * 1024 * 1024 * 4096 = ~4 TB
# PDP-11 Unix orjinalinde uclu dolayli vardi;
# 4K blok boyutuna gecince gerek kalmadi.
# Ama diskler buyudu, triple indirect geri dondu...
                                Unix i-node kapasite hesabi
```

i Blok Boyutu Trade-off'u

Daha küçük blok boyutu: daha az iç parçalanma (internal fragmentation), ancak daha büyük i-node ve daha çok dolaylı erişim gerektirir. Daha büyük blok boyutu: daha hızlı erişim, ancak küçük dosyalarda boş alan israfı.

3.4 Tam I-Node Yapisi Diyagrami

I-Node Alanı	Hedef	Kapasite	Erisim
direct[0..9]	Dogrudan veri bloku	40 KB	1 disk erisimi
single_indirect	Adres listesi blogu	~4 MB	2 disk erisimi
double_indirect	2 katman adres listesi	~4 GB	3 disk erisimi
triple_indirect	3 katman adres listesi	~4 TB	4 disk erisimi

4. Dosya Acma (open) — Adım Adım

Bir dosyayı açmak (`open()`) işletim sistemi içinde karmaşık bir süreç başlatır. Bu sürecin her adımını anlamak, dosya sistemi mimarisini kavramak için kritiktir.

Adım 1: Yol Çözümleme (Path Resolution)

Pathname'deki her bileşen dizin girişi olarak aranır. / mutlak yol için kök dizinden, aksi halde mevcut çalışma dizininden başlanır.

```
open('/usr/ast/mbox', O_RDONLY)
```

```
1a. Kök dizinin i-node'unu oku (i-node #1 veya superblock'tan bul)
```

```
1b. Kök dizinde 'usr' girişini ara -> i-node #6
```

```
1c. i-node #6'nin bloklarını oku -> 'ast' girişini ara -> i-node #26
```

```
1d. i-node #26'nin bloklarını oku -> 'mbox' girişini ara -> i-node #60
```

Adım 2: I-Node Belleğe Yükleme

Hedef dosyanın i-node'u (örn. #60) disk üzerindeki i-list'ten RAM'e okunur. Referans sayacı (reference count) artırılır. Aynı dosya zaten açıksa i-node zaten bellekteydi — tekrar okunmaz.

Adım 3: Dosya Tablosu Girişi (File Table Entry)

Çekirdek içinde bir dosya tablosu (kernel file table) tutulur. Bu dosya için yeni bir giriş oluşturulur: mevcut ofset (current offset = 0), erişim modu (read/write), i-node işaretçisi.

Adım 4: Dosya Tanımlayıcı (File Descriptor)

Dosya tablosundaki girişin indeksi, uygulamaya dosya tanımlayıcı (file descriptor, fd) olarak döndürülür. Bu, açık dosyaya kısa referans olarak kullanılır.

```
fd = open('/usr/ast/mbox', O_RDONLY);
```

```
/* fd = 3 (0=stdin, 1=stdout, 2=stderr zaten açık) */
```

i Evrensel Kavram: File Handle

Neredeyse tüm işletim sistemleri 'file handle' kavramına sahiptir: açık bir dosyaya kısa yoldan başvuru. Unix'te bu fd (integer), Windows'ta HANDLE, Java'da FileInputStream nesnesidir.

5. Dosya Olusturma (create)

Dosya oluşturmak, açmaktan daha fazla adım içerir çünkü hem izin hem de i-node manipülasyonu gerektirir.

```
/* Dosya olusturma adimlari */
Adim 1: Dizin girişi ara
-> Dosya zaten varsa: aç (truncate bayragi varsa icerigini sifirla)
Adim 2: Dizin girişi yok -> yeni giriş oluşturun
-> Dizin dosyasına yazma (normal dosya yazmaya benzer)
-> Bos i-node numarası bul (link count == 0 olan i-node)
Adim 3: I-node numarası 0 olan dizin girişleri bos/silinmistir
-> Yeni i-node tahsis et ve baslat
Adim 4: Dizin girişine yeni i-node numarası yaz
/* Özet: dosya adı -> i-node numarası bağlantısı kurulur */
```

+ Ayrisma (Decoupling)

Dizin girişi ile i-node ayrı tutulduğu için aynı i-node'a birden fazla izin girişi işaret edebilir — bu hard link'in temelidir.

6. Dosya Okuma (read)

Her `read()` çağrısında işletim sistemi şu adımları izler:

A di m	İşlem	Detay
1	Offset -> blok numarası	$\text{current_offset} / \text{block_size} = \text{hangi blok}$, kalan = blok içi konum
2	Bloğu diskten oku	I-node'daki adres listesine bak; gerekirse dolaylı blok
3	Verileri kullanıcıya ilet	Doğru byte aralığını user buffer'a kopyala
4	Offseti güncelle	$\text{current_offset} += \text{okunma miktarı}$
5	Read-ahead (opsiyonel)	Ardisik erişim tespit edilirse bir sonraki bloku önceden iste

```
/* Örnek: 4KB blok, offset=9000'den 100 byte oku */
blok_no = 9000 / 4096 = 2 (3. blok, 0-indeksli)
blok_ici = 9000 % 4096 = 808 (blok içinde başlangıç)
/* blok 2: doğrudan blok (direct[2]) -> tek disk erişimi */
buffer = disk_read(i_node.direct[2])
user_data = buffer[808 .. 907] /* 100 byte */
current_offset = 9100
```

+ Read-Ahead Optimizasyonu

Read-ahead (ondan okuma): Eğer ardışık bloklar okunuyorsa, OS bir sonraki bloğun disk okumasını başlatır ama beklemez. Bu spekülatif işlem, disk gecikmesini kullanıcıdan gizler ve gerçek dünyada belirgin performans artışı sağlar.

7. Dosya Yazma (write)

Yazmak okumaktan daha karmaşıktır çünkü dosya büyüyebilir ve yeni bloklar tahsis edilmesi gerekebilir.

```
/* write() adım adım */
Durum A: Mevcut bloğun ORTASINA yazma
-> 0 bloğu önce diskten oku (read-modify-write)
-> Kullanıcı verilerini buffer'a kopyala
-> Buffer'i 'dirty' olarak işaretleme (yazıyla güncellendi)
Durum B: Yeni blok gerektiren yazma (dosya uzuyor)
-> Freelist'ten boş blok tahsis et
-> I-node'a bu yeni blok adresini ekle
-> Veriyi yeni bloğa yaz
Her iki durumda da:
-> Offset güncelleme: current_offset += yazılan miktar
-> I-node'u dirty işaretleme (boyut değişti, mtime güncellendi)
```

! Dirty Buffer ve Veri Kaybı Riski

Dirty buffer: bellekteki blok diskteki versiyonundan farklıdır. OS periyodik olarak dirty blokları diske yazar (flush/sync). Güç kesilirse dirty buffer'lar kaybolabilir — journaling bu sorunu çözer.

8. Seek — Konum Degistirme

Seek işlemi son derece basittir: yalnızca bellekteki **mevcut byte offset** değeri değiştirilir. Disk kolunu fiziksel olarak hareket ettirmez.

```
lseek(fd, 0, SEEK_SET); /* başlangıca dön */
lseek(fd, 100, SEEK_CUR); /* 100 byte ileri git */
lseek(fd, -50, SEEK_END); /* dosya sonundan 50 byte geri */
/* Seek sadece file_table.current_offset değerini değiştirir */
/* Disk hareketi YOKTUR */
```


i Neden Seek Disk'i Hareket Ettirmez?

Cok gorevli bir sistemde disk kolunu seek aninda hareket ettirmek anlamsizdir: bir sonraki read() geldiğinde başka bir süreç diski kullanıyor olabilir. Gerçek disk erişimi sadece read/write sırasında olur.

9. Dosya Kapatma (close)

```
/* close(fd) adim adim */
Adim 1: I-node referans sayacini azalt (reference count--)
Adim 2: File table girisini sil
Adim 3: Eger i-node link count == 0 ise:
-> Dosya silinmis ama hala acikti
-> Tum bloklari freelist'e iade et
-> I-node'u serbest birak
Adim 4: Baska islem gerekmez — otomatik
```

! Unlink + Close = Gerçek Silme

Acik bir dosyayi silebilirsiniz (unlink)! Dosya yalnızca son close() cagrisinda gercekten silinir. Bu ozellik Unix'te cok kullanilir: gecici dosyalari acip hemen unlink edin — program kapaninca otomatik silinir.

10. Link (Hard Link) Olusturma

Bir hard link oluşturmak, aynı i-node'a işaret eden yeni bir dizin girişi eklemek demektir. İki dosya adı, **aynı veriyi paylaşır** — tek bir i-node.

```
/* link() sistem cagrisi */
link('/home/alice/report.txt', '/home/bob/report.txt')
Adim 1: /home/bob/ dizinine yeni giriş ekle
-> isim: 'report.txt'
-> i-node numarasi: AYNI i-node (ornegin #42)
Adim 2: I-node #42'nin link count degerini artir
-> link_count = 2 (iki isim, ayni dosya)
/* Artik iki dosya adi var, bir i-node var, bir veri kopyasi var */
```

Ozellik	Hard Link	Symbolic Link (Soft Link)
I-node	Aynı i-node'u paylasır	Ayrı i-node; hedef yolu içerir
Hedef silinirse	Veri kaybolmaz (link count>0)	Dangling link — geçersiz
Disk bolumu	Aynı bolumde olmalı	Farklı bolum veya dosya sistemi olabilir
Dizin için	Genellikle izin verilmez	İzin verilir
stat() görünüm	Gerçek dosya gibi	Sembolik link olarak görünür

11. Unlink — Dosya Silme

Unix'te dosya silme aslında bir 'unlink' işlemidir — dizin girişi kaldırılır ve i-node'daki link sayacı azaltılır.

```
/* unlink() adım adım */
Adım 1: Dizin girişini bul ve sil (i-node numarası 0 yap)
Adım 2: I-node link_count--
if link_count > 0:
/* Baska isimler var; dosya hala yasıyor. Bir sey yapma. */
elif link_count == 0 AND reference_count == 0:
/* Kimse acmamis ve isim kalmamis -> gercek silme */
-> Tum bloklari freelist'e iade et
-> I-node'u serbest birak (i-list'te bos olarak isaretleme)
elif link_count == 0 AND reference_count > 0:
/* Acik ama isimsiz -> process kapatınca silinecek */
-> Bekle...
```

! Crash + Acik-Silinmis Dosya

Sistem crash olursa ve link_count=0 ama reference_count>0 olan bir dosya varsa ne olur? Yeniden başlatmada fsck bu tutarsızlığı bulur ve dosyayı /lost+found dizinine taşır.

12. Metadata Guncelleme

Her dosya işlemi belirli zaman damgalarını ve diğer metadata alanlarını günceller:

Islem	Guncellenen Alan	Not
read()	atime (son erisim zamanı)	Performans icin disable edilebilir (noatime)
write()	mtime (son degisiklik zamanı)	Dosya icerigi degisti
chmod/chown	ctime (i-node degisiklik zamanı)	mtime degismiyor
link/unlink	ctime	Link count degisti
read+write	ctime de guncellenir	I-node dirty isaret edilir

Bellekteki i-node **dirty** olarak işaretlenir. Periyodik olarak (örneğin update daemon veya sync çağrısıyla) dirty i-node'lar diske yazılır.

! atime Performans Sorunu

atime (access time) her okumada guncellenmesi ciddi performans yuku olusturabilir cunku her read() icin bir de disk yazisi gerekir. Bu nedenle modern sistemler noatime veya relatime mount secenegi kullanir.

13. Dizin Olusturma ve Silme

13.1 Dizin Olusturma (mkdir)

```
/* mkdir() adim adim */
Adim 1: Yeni i-node tahsis et (dosya olusturmayla ayni)
Adim 2: Cekirdek . ve .. girislerini YAZ
. -> bu dizinin i-node numarasi
.. -> ust dizinin i-node numarasi
Adim 3: Ust dizinin link_count++ (.. ona isaret ediyor)
/* Kullanici programlari write() ile dizine DOGRUDAN yazamazlar */
/* Sadece ozel sistem cagrisi (mkdir) kullanilir */
```

13.2 Dizin Silme (rmdir)

```
/* rmdir() adim adim */
On Kosul: Dizin . ve .. haricinde BOS olmalı
-> Bos degilse: ENOTEMPTY hatasi don
Adim 1: Normal dosya silme gibi isleme (unlink benzeri)
Adim 2: Ust dizinin link_count-- (.. artik yok)
/* Ilginc not: Gecerli calisma dizini (cwd) silinebilir! */
/* Hatta ust dizini de silebilirsiniz... */
```

! Silinmiş Dizinde Olmak

Mevcut çalışma dizinini silebilirsiniz! Bu durumda siz hala o dizindeyken (reference count > 0) dizin 'var' gibi davranır; ama `ls ..` gibi komutlar garip sonuçlar verebilir. Son `close/chdir`'den sonra gerçekten silinir.

14. Rename — Yeniden Adlandırma

```
/* rename() adım adım */
Adım 1: Yeni isimle HARD LINK oluşturun (eski i-node'a işaret eder)
Adım 2: Eski isimle UNLINK yap
/* Bu adımlar çekirdek tarafından atomik biçimde yapılmalıdır */
/* Neden? Bir dizine hard link oluşturmak gerekebilir */
/* Kullanıcı programı bunu yapamaz; çekirdek yapar */
/* rename('/old/path', '/new/path') */
```

i Neden Çekirdekte?

`rename()` çekirdek içinde yapılmalıdır çünkü birinci adım (dizine hard link oluşturma) kullanıcı programlarına yasaktır. Eğer ikinci adım (eski ismi sil) sistem çökmesi nedeniyle yapılamazsa dosyanın iki ismi olur — `fsck` bunu düzeltir.

15. Dosya Sistemi Bileşenleri

Bir Unix dosya sistemi bölümü (partition) dört ana bölgeden oluşur:

```
Disk Bölümünün Düzeni (Layout):
+-----+-----+-----+-----+
| Boot Record | Superblock | I-List | Data Area |
| (MBR)      | | (I-Node | (Dosya içeriği, dizinler) |
| | | dizisi) | |
+-----+-----+-----+-----+
Blok 0 Blok 1 Blok 2..N Blok N+1..
Klasik Unix dosya sistemi disk düzeni şeması
```

15.1 Boot Record (MBR)

- Diskin en başında (blok 0) yer alır
- BIOS tarafından bilgisayar açılışında okunur
- Hangi bölümden boot edileceği bilgisini tutar
- Disk bölümlendirme tablosunu (partition table) içerir

15.2 Superblock

- Dosya sisteminin kimliği ve temel parametreleri
- I-list'in nerede başladığı ve boyutu
- **Clean shutdown flag**: son kapatma temiz miydi? (fsck kararı için)
- Serbest blok sayısı, serbest i-node sayısı
- Blok boyutu, dosya sistemi boyutu

x Superblock Bozulması

Superblock çok kritiktir. Bozulursa tüm dosya sistemine erişim kaybedebilirsiniz. Bu nedenle modern dosya sistemleri superblock'u birçok kopyada saklar.

15.3 I-List (I-Node Dizisi)

- Klasik Unix: superblock'tan hemen sonra sirasiyla bloklar
- Modern Unix (FFS/ext4): her cylinder group'ta bir i-list parçası (dağıtılmış)
- I-node numarası = bu dizideki indeks
- Cylinder group yapısı: yerellik sağlar, uzun seek'lerden kaçınar

```
/* Cylinder Group yapisi */
Her cylinder group icerir:
- I-list parçası (grup içindeki dosyalar için i-node'lar)
- Freelist (gruptaki boş bloklar için bitmap)
- Data area (gruptaki veri blokları)
/* Yeni dosya -> aynı cylinder group'ta blok tahsisi (yerellik!) */
/* Daha sonraki bloklar -> başka cylinder group'larda */
```

15.4 Data Area

- Disk alanının büyük çoğunluğu buradadır
- 4-8 KB boyutunda bloklar: büyük dosyalar için
- Kalan parçalar (fragments): 512-byte sub-bloklar, küçük dosyalar için
- Cift blok boyutu: hem RAM/disk tasarrufu hem de verimlilik

i Fragment Mantığı

Neden fragment? 8 KB blokta 100 byte saklarsak 7.9 KB israf ederiz. Fragment'lar bu problemi çözer: son kısmı 512-byte parçalara bölünmüş bloklarda sakla. Büyük dosyalar tam blok kullanır, küçük dosyalar fragment kullanır.

16. Windows FAT Dosya Sistemi

FAT (File Allocation Table), DOS ve erken Windows sürümlerinin dosya sistemidir. Basit ve geniş donanım desteğine sahiptir.

Ozellik	FAT12	FAT16	FAT32
Max dosya	32 MB	2 GB	4 GB
Max bolum	32 MB	2 GB	2 TB
Adres bits	12 bit	16 bit	28 bit (4 bit rezerv)
Hedef	Floppy	Eski disk	USB, SD kart, Windows 9x

16.1 FAT Yapisi

```
FAT = File Allocation Table
Her giriş: bir sonraki bloğun adresi (linked list, ama RAM'de!)
Blok 0 -> 3 (bloğun devamı blok 3'te)
Blok 3 -> 7 (devam blok 7'de)
Blok 7 -> 9 (devam blok 9'da)
Blok 9 -> EOF (son blok)
/* Avantaj: Disk okumadan blok zinciri takip edilir */
/* Dezavantaj: Tüm FAT RAM'de olmalı -> büyük diskler = büyük RAM */
/* Örnek: 200 GB disk, 1 KB blok, 4 byte/giriş */
Blok sayısı = 200 * 1024 * 1024 = 209,715,200
FAT boyutu = 209,715,200 * 4 = ~800 MB RAM gerekir!
```

16.2 FAT Dizin Yapisi

- Kok dizin (root directory) sınırlı boyuta sahiptir (FAT16: max 512 giriş)
- Dosya adları 8.3 formatında (8 karakter + 3 karakter uzanti)
- Metadata (boyut, tarih, özellik) dizin girişinde saklanır (i-node yok!)
- Dizin girişi doğrudan FAT tablosundaki ilk bloğu gösterir

Alan	Boyut (byte)	Aciklama
Dosya adı	8	Boslukla doldurulmus, buyuk harf
Uzanti	3	Boslukla doldurulmus
Ozellikler	1	ReadOnly, Hidden, System, Directory, Archive
Rezerv	10	Uzun ad desteği için kullanilir (LFN)
Son degisiklik saati	2	Saat:Dakika:Saniye (2sn hassas)
Son degisiklik tarihi	2	Yil:Ay:Gun
Ilk FAT girişi	2	Dosyanın ilk blok numarası
Dosya boyutu	4	Byte cinsinden

x FAT'ta I-Node Yok

FAT'ta i-node yoktur! Tum metadata (boyut, tarih, izinler) dizin girişinde saklanır. Bu hard link'i imkansız kılar ve metadata senkronizasyonunu zorlaştırır. Dizin girişi silinirse metadata da kaybolur.

17. FAT'ta Uzun Dosya Adlari (Long File Names)

Windows 95/98 ile birlikte FAT16/32'de 8.3 sınırını aşan uzun dosya adı desteği geriye dönük uyumlulukla eklendi.

Cozum Yontemi: Sahte Dizin Girisler

```

Uzun ad: 'My Long Document Name.txt'
8.3 kisaltmasi: 'MYLONG~1.TXT' (otomatik uretilir)
Dizin girisleri:
[LFN giriş 3] '...ame.txt' <- uzun adının son parçası
[LFN giriş 2] 'ument N' <- orta parça
[LFN giriş 1] 'My Long Doc' <- ilk parça
[GERCEK GIRIS] 'MYLONG~1.TXT' <- esas giriş (metadata burada)
/* LFN girisleri gecersiz gibi gozukur (special ozellik biti = 0x0F) */
/* Eski DOS bu girisleri gorunmez/atlar */
/* Her LFN girişinde checksum var: esas giriş silinirse gecersiz isaretlenir */

```

Ozellik	Aciklama
Max uzun ad uzunlugu	255 Unicode karakter
Karakter kümesi	UTF-16 (Unicode)
Geriye uyumluluk	Eski DOS LFN girişlerini atlar (0x0F özellik biti)
Checksum	Kisa ad silinirse LFN girişleri gecersiz sayilir
Dizin girişi sayısı	1 gerçek + N sahte (13 karakter/giriş)

! Geriye Donuk Uyumluluk Bedeli

Bu çözüm zarif değil ama işe yarıyor. Gerçek dosya sistemi tasarımcıları geriye dönük uyumluluğun ne kadar çılgınca kısıtlamalara yol açabileceğini bu örnekten görebilir.

18. Disk Dump ve Yedekleme (Backup)

Yedekleme iki temel senaryoya karşı koruma sağlar:

- **Disaster recovery:** Disk arızası, yangın, sel vb.
- **Stupidity recovery:** Kullanıcının yanlışlıkla dosya silmesi

18.1 Fiziksel Dump vs. Mantıksal Dump

Yontem	Tanim	Avantajlar	Dezavantajlar
Fiziksel Dump (Physical)	Blok 0'dan başla, tüm blokları sırayla yaz	Basit, hızlı, bire bir	Bos blokları da yazar; artissal dump imkansız; bozuk bloklar sorun
Mantıksal Dump (Logical)	Bir veya daha fazla dizinden başla, dosyaları ozyinelemeli yaz	Artissal dump, bos blokları atla, seçici restore	Daha karmaşık, bağlantı (link) sorunları

18.2 Mantıksal Dump Algoritması (4 Asamalı)

Unix'te kullanılan mantıksal dump algoritması 4 aşamada çalışır:

Aşama 1: İşaretleme — Degismis Dosyalar ve Tüm Dizinler

Tüm degismis dosyalari ve her dizini (degismis olmasa bile) isaretleme. Bitmap'te her i-node için bir bit tutulur.

```
for each i-node:
    if dosya ve son dump'tan beri degistiyse:
        mark(i-node) # modified files
    if dizin ise:
        mark(i-node) # ALL directories
```

Aşama 2: Eleyerek Isaretleme — Bos Dizinleri Cikar

İçinde hiç isaretili dosya olmayan dizin isaretin kaldırılır. Bu adım bottom-up çalışır.

```
for each directory (bottom-up):
    if hiç isaretili alt dosya/dizin yoksa:
        unmark(directory) # gerekmez, atla
```

Aşama 3: Dizinleri Yaz

I-node numarasına göre sıralanmış isaretili dizinleri tape'e yaz. Restore sırasında dizin yapısı önce oluşturulacak.

Aşama 4: Dosyalari Yaz

Isaretili dosyalari (dizinler haric) i-node numarasına göre sırayla tape'e yaz.

18.3 Kurtarma (Recovery) Algoritması

```
Adım 1: Disk üzerinde bos dosya sistemi oluşturun
Adım 2: En son tam dump'ı (level 0) yükleyin
-> Önce dizinleri yükleyin
-> Sonra dosyaları yükleyin
Adım 3: Her artısal dump'ı sırayla yükleyin (level 1, 2, ...)
/* Sorunlar */
Serbest bloklar: Yüklemeden sonra yeniden hesapla
Hard link'ler: İki dizinde aynı i-node -> restore önce yapılır
```

18.4 Artissal (Incremental) Yedekleme Seviyeleri

Seviye	Ne Yazar?	Ornek Sik liđi
Level 0	Her seyi (tam yedek)	Haftada bir veya ayda bir
Level 1	Son level 0'dan bu yana degisenleri	Gunluk
Level 2	Son level 1'den bu yana degisenleri	Saatlik

i Dump'in Dosya Sistemi API'si Atlamasi

Dosya sistemi dump'i Unix'te dogrudan dosya sistemi API'si uzerinden yapilamaz: 'delik'leri (holes — sparse file bosluklari) atlamak için dogrudan i-node bazlı okuma gerekir. Restore ise normal dosya sistemi uzerinden yapılabilir.

19. Quiz Sorulari ve Cevaplari

i Kullanım Rehberi

Aşağıdaki sorular sınav formatındadır. Cevapları okumadan önce kendiniz çözmeye çalışın.

S1. Unix'te bir i-node 10 doğrudan blok, 1 tekli dolaylı, 1 çiftli dolaylı ve 1 uçlu dolaylı blok adresine sahiptir. Blok boyutu 4 KB ve adres boyutu 4 byte olduğunu varsayarsak, bu i-node ile erişilebilecek maksimum dosya boyutunu hesaplayınız. Her adres kademesi için kapasite ve disk erişim sayısını ayrı ayrı belirtiniz.

-> Her blokta kaç adres? $4096 / 4 = 1024$ adres

-> Doğrudan: $10 \times 4096 = 40,960$ byte (~40 KB), 1 disk erişimi

-> Tekli dolaylı: $1024 \times 4096 = 4,194,304$ byte (~4 MB), 2 disk erişimi

-> Çiftli dolaylı: $1024^2 \times 4096 = 4,294,967,296$ byte (~4 GB), 3 disk erişimi

-> Uçlu dolaylı: $1024^3 \times 4096 = 4,398,046,511,104$ byte (~4 TB), 4 disk erişimi

-> Toplam maksimum: $40 \text{ KB} + 4 \text{ MB} + 4 \text{ GB} + 4 \text{ TB} \approx 4 \text{ TB}$

-> (Doğrudan ve tekli dolaylı toplam ihmal edilebilir düzeydir)

S2. 'open()' sistem çağrısı ile 'read()' sistem çağrısı arasındaki süreçte i-node ne zaman belleğe yüklenir, ne zaman bellekten çıkarılır? File descriptor (fd), file table entry ve i-node arasındaki ilişkiyi açıklayın.

-> I-node belleğe yükleme: open() çağrısında — i-node numarası bulunduğunda i-list'ten RAM'e okunur, referans sayacı artırılır.

-> I-node bellekten çıkarma: son close() çağrısında — referans sayacı 0 olduğunda (ve link count > 0 ise) dirty ise diske yazılıp serbest bırakılır.

-> İlişki:

-> fd (integer) -> proses fd tablosu -> kernel file table entry

-> file table entry -> i-node isaretçisi + current offset + erişim modu

-> I-node -> disk blok adresleri + metadata

-> Önemli: Aynı dosyayı iki kez open() eden iki proses -> iki ayrı file table entry -> aynı i-node. Her prosesin bağımsız offset'i vardır.

S3. Unix'te bir dosya açıkken silinirse (unlink) ne olur? Sistem bu sırasında crash olursa ne olur? fsck bu durumu nasıl kurtarır?

-> Açıkken unlink: Dizin girişi silinir (link_count--). Eğer link_count=0 olursa ama reference_count>0 ise dosya gerçekten silinmez — son close()'a kadar varlığı devam eder.

-> Crash olursa: link_count=0 ama reference_count>0 olan dosya tutarsız kalır. I-node 'kullanılmıyor' gibi görünür ama blokları hala işaret ediliyor.

-> fsck kurtarması: Başlangıçta fsck i-node'ları tarar; link_count=0 ama blok içeren i-node'ları bulur ve bunları /lost+found dizinine koyar (i-node numarasını dosya adı olarak kullanır). Kullanıcı sonra bu dosyaları inceleyip kurtarabilir.

S4. Hard link ile symbolic link (soft link) arasindaki farklari kapsamli sekilde aciklayiniz. Hangi durumlarda hangisi tercih edilir?

-> Hard link: Ayni i-node'a ikinci bir dizin girisidir. Ayni dosya sisteminde olmalidir. Hedef silinse bile veri kaybolmaz ($\text{link_count} > 0$). Dizinlere hard link genellikle yasaktir (dongu riski). `stat()` ile ayirt edilemez.

-> Symbolic link: Ayri bir i-node, icerigi hedef yolunun metnidir. Farkli dosya sistemlerine veya dizinlere isaret edebilir. Hedef silinirse 'dangling link' (sarkan baglanti) olusur. `stat()` ile ayirt edilebilir.

-> Tercih:

-> Hard link: Ayni veriyi bircok yerde gosterme, yedeksiz erasure-safe baglanti isteniyor

-> Symbolic link: Farkli disk/FS arasi baglantilar, dizin baglantilari, kiralik yollar (`/usr/bin/python -> /usr/bin/python3`)

S5. FAT dosya sisteminde buyuk bir disk ile kullanmanin neden RAM kullanimi acisindan sorunlu oldugunu hesaplayin: 200 GB disk, 1 KB blok boyutu, 4 byte FAT girisi. FAT ne kadar RAM gerektirir?

-> Blok sayisi = $200 \text{ GB} / 1 \text{ KB} = 200 * 1024 * 1024 = 209,715,200$ blok

-> FAT boyutu = $209,715,200 * 4 \text{ byte} = 838,860,800$ byte

-> $= 838,860,800 / (1024 * 1024) \approx 800 \text{ MB RAM}$

-> Sonuc: Sadece FAT tablosu icin 800 MB RAM gerekir — bu modern sistemlerde bile kabul edilemez bir israftir. Unix i-node yaklasimi sadece acik dosyalarin i-node'larini bellekte tutar; cok daha verimlidir.

S6. Mantiksal dump algoritmasının 4 asamasını açıklayın. Neden sadece değişmiş dosyaları işaretlemek yeterli değil, tüm dizinler de işaretlenmelidir?

- > Asama 1: Tüm değişmiş dosyaları VE tüm dizinleri işaretleme (değişmiş olsun ya da olmasın)
- > Asama 2: Hiç işaretli dosyası olmayan dizinlerin işaretini kaldır (bottom-up)
- > Asama 3: İşaretli dizinleri i-node sırasına göre tape'e yaz
- > Asama 4: İşaretli dosyaları tape'e yaz

- > Neden tüm dizinler?: Kurtarma sırasında dosyanın doğru konuma konması için tüm dizin yapısı restore edilmelidir. Sadece değişmiş bir dosyanın kendisi yeterli değil — ona giden yoldaki tüm dizinler de lazım. Örnek: /a/b/c.txt değiştiyse hem /a hem /a/b hem de c.txt dump edilmeli.

S7. Cylinder group nedir ve Unix dosya sisteminde neden kullanılır? Geleneksel tek merkezi i-list yaklaşımının hangi problemi çözer?

- > Cylinder group: Bir disk bölümünü küçük gruplara ayırır. Her grup kendi i-list parçasını, freelist'ini ve data alanını içerir.

- > Çözülen problem: Geleneksel yöntemde i-list diskin başında, veri alanı sonunda yer alır. Bir dosyayı okumak için önce i-node için başa seek edilir, sonra veriler için sona seek edilir — çift seek.

- > Cylinder group avantajları:

 - > - Yeni dosya aynı cylinder group'ta blok alır -> i-node ve veri bloğu yakın
 - > - Seek süresi azalır (yerellik ilkesi)
 - > - Dosya sistemi fragmantasyonu azalır
 - > - fsck daha hızlı çalışır (bolge bolge kontrol)

S8. Windows FAT'ta uzun dosya adi (LFN) destegi nasil saglandi? Bu cozumun geriye donuk uyumlulugu nasil korunur ve ne gibi riskleri vardir?

-> Yontem: Asil dizin girisinden once 'sahte' LFN giris bloklari eklenir. Her LFN girisi 13 Unicode karakter tutar ve 0x0F ozellik bitine sahiptir.

-> Geriye uyumluluk: Eski DOS bu girisleri gecersiz/okunamaz olarak gorur ve atlar. Asil kısa ad girisi (8.3) tam olarak eski sistemlerin bekledigi bicdedir.

-> Checksum: Her LFN girisi kısa adindan hesaplanmış bir checksum icerir. Eger kısa ad girisi DOS altinda silinip yeniden olusturulursa checksum eslesmeye bozulur, LFN girisleri gecersiz sayilir.

-> Riskler: Dizin tutarsizligi olasiliği, LFN girisleri ve asil giriş arasında sıra bozulmasi (baska arac veya sistem etkilerse), checksum hatasi.

20. Kitap Bolum Sonu Sorulari ve Cozumleri

Asagidaki sorular Tanenbaum 'Modern Operating Systems' kitabinin File Systems bolumunden secilmistir (konu kapsamıyla ilgili olanlar).

Kitap Soru 1

Unix V7 dosya sisteminde, dosya blocklari icin i-node'da 10 dogrudan adres, 1 tekli dolayli, 1 ciftli dolayli blok vardir. Blok boyutu 512 byte, her disk adresi 3 bytedir. Bu dosya sisteminde saklanan maksimum dosya boyutu nedir?

Cozum:

Her blokta adres sayisi = $512 / 3 = 170$ adres (yaklasik)

Dogrudan: $10 \times 512 = 5,120$ byte

Tekli dolayli: $170 \times 512 = 87,040$ byte

Ciftli dolayli: $170 \times 170 \times 512 = 14,796,800$ byte (~14 MB)

Toplam: $5,120 + 87,040 + 14,796,800 \approx 14.88$ MB

Not: PDP-11 Unix'in gercek i-node'unda 8 dogrudan blok vardi; modern hesaplarda 10 kullanilir.

Kitap Soru 2

Blok boyutu 1 KB ve disk adresi 4 byte olan bir dosya sisteminde, maksimum dosya boyutu nedir? Tekli, çiftli ve uclu dolayli blok kullanin.

Cozum:

Her blokta adres = $1024 / 4 = 256$ adres

Dogrudan (10 blok): $10 \times 1024 = 10,240$ byte (~ 10 KB)

Tekli dolayli: $256 \times 1024 = 262,144$ byte (~ 256 KB)

Çiftli dolayli: $256^2 \times 1024 = 67,108,864$ byte (~ 64 MB)

Uclu dolayli: $256^3 \times 1024 = 17,179,869,184$ byte (~ 16 GB)

Toplam: ~ 16 GB + 64 MB + 256 KB + 10 KB ~ 16 GB

Kitap Soru 3

Bir dosyaya iki hard link verilirse ve bu dosya silinirse ne olur? i-node ve disk bloklari ne zaman gercekten serbest birakilir?

Cozum:

Baslangic: `link_count = 2` (iki dizin girisi, ayni i-node)

Birinci unlink: `link_count-- -> link_count = 1`. Sadece ilk dizin girisi silindi. I-node ve bloklar hala var. Dosya ikinci ismiyle hala erisilebilir.

Ikinci unlink: `link_count-- -> link_count = 0`. Ve `reference_count = 0` ise (dosya acik degil):

-> Tum bloklar freelist'e iade edilir

-> I-node serbest birakilir (`link_count=0`)

Eger dosya hala aciksa (`reference_count > 0`): Bloklar henuz serbest birakilmaz; son `close()` aninda serbest birakilir.

Kitap Soru 4

FAT dosya sisteminde (FAT-32) maksimum dosya boyutu neden 4 GB ile sinirlidir? Bu siniri ortadan kaldirmak icin ne degistirilmeli?

Cozum:

FAT-32 dizin girisinde dosya boyutu alanı 32 bit (4 byte) ile sinirlidir:

$$\text{Max boyut} = 2^{32} - 1 = 4,294,967,295 \text{ byte} \approx 4 \text{ GB}$$

Not: FAT adresi 28 bit olsa da (32-bit girisler, 4 bit rezerv), asil sinir dizin girisindeki 32-bit boyut alanindan gelir.

Cozum: Dosya boyutu alanini 64 bit yapmak gerekir. Bu Microsoft'un exFAT (Extended FAT) ile yaptigi tam da budur — exFAT 128 EB'a kadar dosya destekler. Ancak bu geriye donuk uyumlulugu bozar.

Kitap Soru 5

Bir dosya sisteminde ciftli dolayli blok mekanizmasini aciklayan bir diyagram ciziniz (tablo olarak gosterin). 4 KB blok ve 4 byte adres varsayin. Bir dosyanin 100. MB'ina erisim icin kac disk I/O islemi gereklidir?

Cozum:

$100 \text{ MB} = 100 \times 1024 \times 1024 = 104,857,600 \text{ byte}$

$\text{Blok numarasi} = 104,857,600 / 4096 = 25,600. \text{ blok}$

Dogrudan (0-9. blok): 10 blok

Tekli dolayli (10 - 1033. blok): 1024 blok

Ciftli dolayli: $1024^2 = 1,048,576 \text{ blok} \rightarrow \text{blok } 25,600 \text{ buradadir}$

Ciftli dolayli icinde konum:

$\text{offset} = 25,600 - 10 - 1024 = 24,566$

$\text{ciftli_ind_idx} = 24,566 / 1024 = 23 \text{ (24. tek dolayli blok)}$

$\text{tekli_ind_idx} = 24,566 \% 1024 = 934 \text{ (934. veri bloku)}$

Disk I/O sayisi:

1. Ciftli dolayli bloku oku (i-node'dan adres al)

2. 23. tekli dolayli bloku oku

3. Veri blok 934'u oku

$\rightarrow \text{Toplam: 3 disk I/O}$

Kitap Soru 6

Bir Unix sisteminde /etc/passwd dosyasi icin su bilgiler verilmistir: i-node #47, link_count=1, blok boyutu 4KB, dosya boyutu 3210 byte. Bu dosyayi okuyan bir process kapatmadan once delete() cagirirsa ne olur? Process sonra dosyayi okumaya devam edebilir mi?

Cozum:

delete() = unlink() cagrisidir.

Adim 1: /etc/passwd dizin girisi silinir (i-node numarasi 0 yapilir).

Adim 2: i-node #47'nin link_count-- -> link_count = 0.

Adim 3: reference_count > 0 (process hala acik tuttu) -> BLOKLAR HENUZ SERBEST BIRAKILMAZ.

Process ne gorur?

- Process kendi file descriptor'ini kullanarak okumaya DEVAM EDEBILIR.
- Diger processler /etc/passwd'i acamaz (dizin girisi yok).
- Dosya sanal olarak var ama isimsiz.

Ne zaman gercekten silinir?

-> Process close() yapinca reference_count=0 ve link_count=0 -> bloklar serbest, i-node serbest.

Ozet — Anahtar Kavramlar

Kavram	Ozet
Multi-level Indirection	direct[10] + single/double/triple indirect. Kucuk dosya = hizli, buyuk dosya = katmanli erisim.
open()	Yol cozumleme -> i-node belleğe yikle -> file table girisi olustur -> fd dondur.
read()	offset/block_size = blok no -> disk oku -> kullaniciya ver -> offseti guncelle. Read-ahead ile optimizasyon.
write()	Mevcut blok ortasina yazma: once oku. Yeni blok: freelist'ten tahsis + i-node guncelle.
seek()	Sadece file table'daki offset degisir. Disk hareketi YOKTUR.
close()	reference_count--. Eger link_count=0 ve ref=0 -> gercek silme. Dirty i-node diske yaz.
link/unlink	link: dizin girisi ekle + link_count++. unlink: dizin girisi sil + link_count--. 0 olunca blok iade.
FS Bilesenleri	Boot record + Superblock + I-List (cylinder groups) + Data Area (bloklar + fragments).
Windows FAT	I-node yok, metadata dizin girisinde. FAT tablosu RAM'de = buyuk diskler = buyuk RAM.
LFN	Sahte 0x0F ozellik bitli girisler. Eski DOS'un goremez. Checksum ile tutarlilik.
Logical Dump	4 asama: isaretleme -> eleme -> dizin yazma -> dosya yazma. Artissal seviyeli yedekleme.